
PRISM

Formal Modeling and Analysis

This material is adapted with permission from

- Dave Parker (DP), “Probabilistic Model Checking”, slides for a lecture course taught in the Department of Computer Science at Oxford University (2011) <http://www.prismmodelchecker.org/lectures/pmc/>
- Accompanying practical course (PRISM tutorial)
<http://www.prismmodelchecker.org/courses/pmc1112/>
<http://www.prismmodelchecker.org/tutorial/>

Formal Methods

- Software engineering methods can be divided into:
 - semi-formal methods (simple notations based on combinations of diagrams and text)
 - formal methods
- **Formal methods** are mathematically based techniques for specification, design and verification
- Formal methods are useful in software engineering when
 - The problem to be solved is novel or difficult
 - The focus is on **quality** attributes: **reliability, performance**

Probabilistic Model Checking

- **Probabilistic model checking** is formal technique for the modeling and analysis of stochastic systems
- **PRISM** is a leading tool in probabilistic model checking
 - It incorporates state-of-the art structures and algorithms
 - **Application domains**: distributed systems, communication protocols, randomized algorithms, biological systems
- PRISM is a **state-oriented notation**
 - The execution history does not influence the next state
 - The next state depends only on the current state and some current stimuli (or event)

Probabilistic Model Checking

- **PRISM** is based on the following formalisms (including extensions with costs and rewards):
 - **DTMC** (Discrete-Time Markov Chains)
 - **CTMC** (Continuous-Time Markov Chains)
 - **MDP** (Markov Decision Processes)
 - **PTA** (Probabilistic Timed Automata)
- Queries can be expressed in a **property specification language** incorporating the temporal logics **PCTL**, **CSL**, **LTL** and **PCTL*** (including extensions for costs/rewards)

Motivation

- Complexity of model checking
 - generally polynomial in model size (number of states)
- State space explosion problem
 - models for realistic case studies are typically huge
- Clearly (efficient) tool support is required
- Benefits:
 - fully automated process
 - high-level languages/formalisms for building models
 - visualisation of quantitative results

Probabilistic model checkers

- PRISM (this lecture): DTMCs, MDPs, CTMCs, PTAs + rewards
- Markov chain model checkers
 - MRMC: DTMCs, CTMCs + reward extensions
 - PEPA toolset: CTMCs + CSL
- Markov decision process (MDP) tools
 - LiQuor: LTL verification for MDPs (Probmela language)
 - RAPTURE: prototype for abstraction/refinement of MDPs
 - ProbDiVinE: parallel/distributed LTL model checking of MDPs
- Simulation-based probabilistic model checking:
 - APMC, Ymer (both based on PRISM language), VESTA
- And more
 - APNN-Toolbox, SMART, CADP, Möbius, PASS, PARAM, ...
 - see: <http://www.prismmodelchecker.org/other-tools.php>

The PRISM tool

- **PRISM: Probabilistic symbolic model checker**
 - developed at Birmingham and Oxford University, since 1999
 - free, open source (GPL)
 - versions for Linux, Unix, Mac OS X, Windows, 64-bit OSs
- **Modelling of:**
 - DTMCs, CTMCs, MDPs, PTAs + costs/rewards
- **Model checking of:**
 - PCTL, CSL, LTL, PCTL* + extensions + costs/rewards

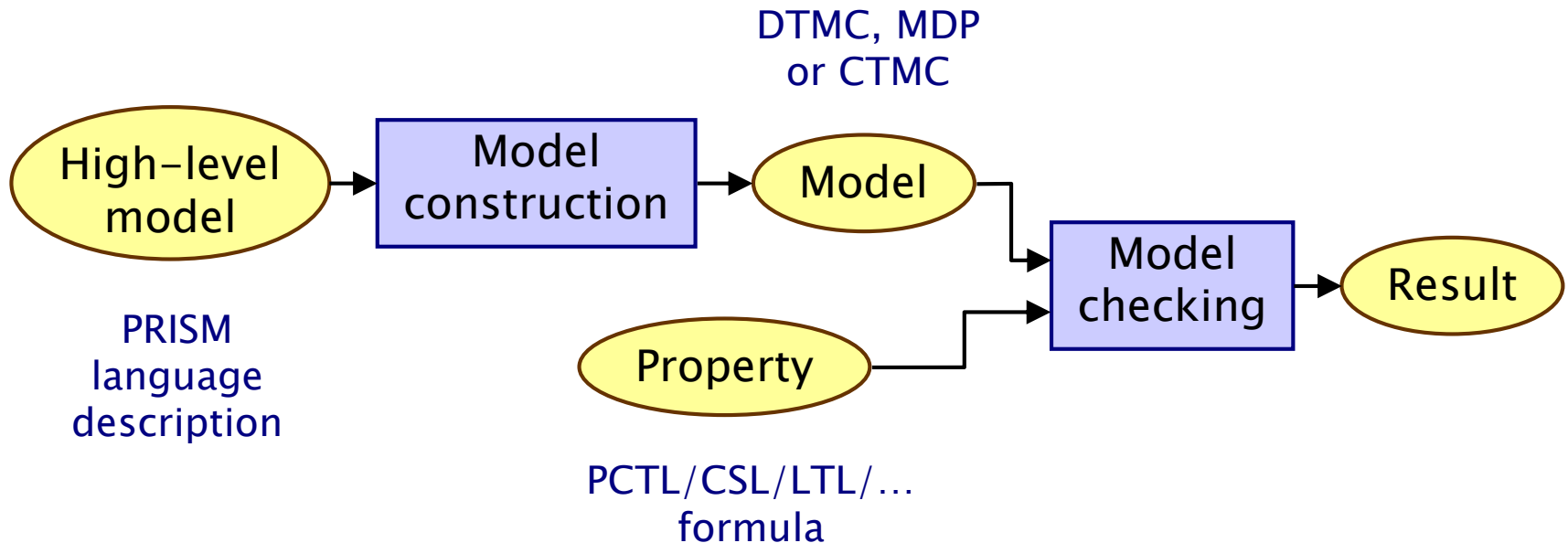


PRISM functionality

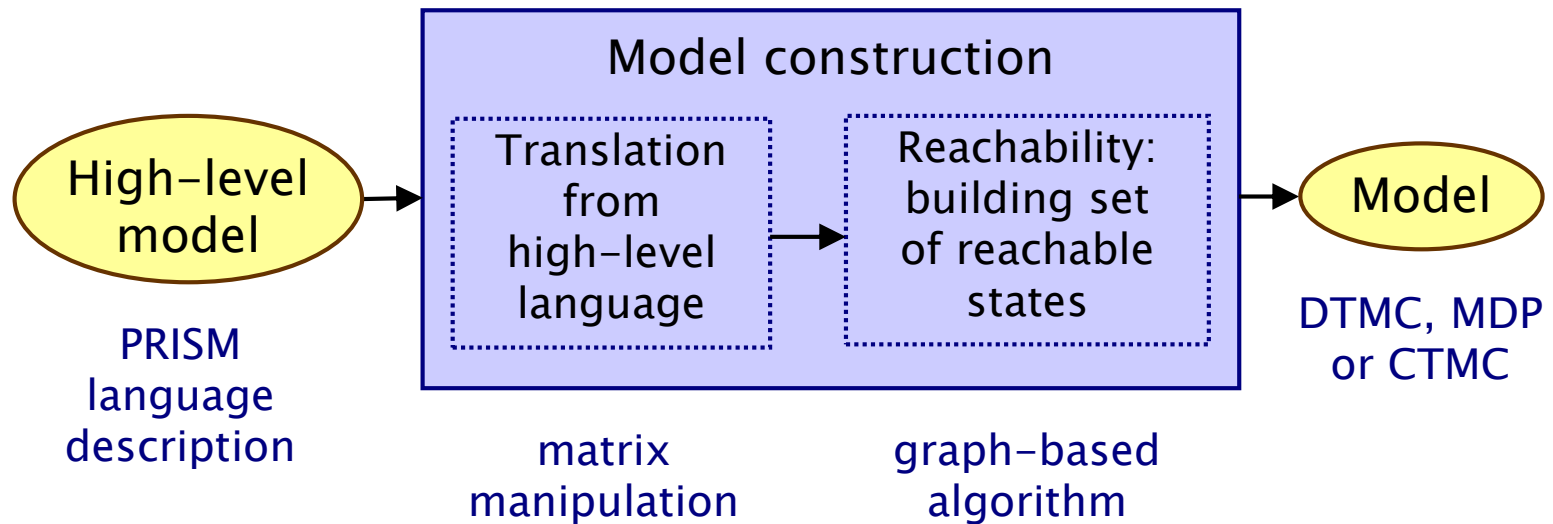
- High-level modelling language
- Wide range of model analysis methods
 - efficient symbolic implementation techniques
 - also: approximate verification using simulation + sampling
- Graphical user interface
 - model/property editor
 - discrete-event simulator – model traces for debugging, etc.
 - easy automation of verification experiments
 - graphical visualisation of results
- Command-line version
 - same underlying verification engines
 - useful for scripting, batch jobs

Probabilistic model checking

- Overview of the probabilistic model checking process
 - two distinct phases: **model construction**, **model checking**



Model construction



Modelling languages/formalisms

- Many high-level modelling languages, formalisms available
- For example:
 - probabilistic/stochastic process algebras (PEPA, Stochastic Pi)
 - stochastic Petri nets
 - stochastic activity networks
- Custom languages for tools, e.g.:
 - PRISM modelling language
 - Probmela (probabilistic variant of Promela, the input language for the model checker SPIN) – used in LiQuor

PRISM modelling language

- Simple, textual, state-based language
 - modelling of DTMCs, CTMCs, MDPs, ...
 - based on Reactive Modules [AH99]
- Basic components...
- Modules:
 - components of system being modelled
 - composed in parallel
- Variables
 - finite (integer ranges or Booleans)
 - local or global
 - all variables public: anyone can read, only owner can modify

PRISM modelling language

- Guarded commands
 - describe behaviour of each module
 - i.e. the changes in state that can occur
 - labelled with probabilities (or, for CTMCs, rates)
 - (optional) action labels

$[send] (s=2) \rightarrow p_{loss} : (s'=3) \& (lost'=lost+1) + (1-p_{loss}) : (s'=4);$



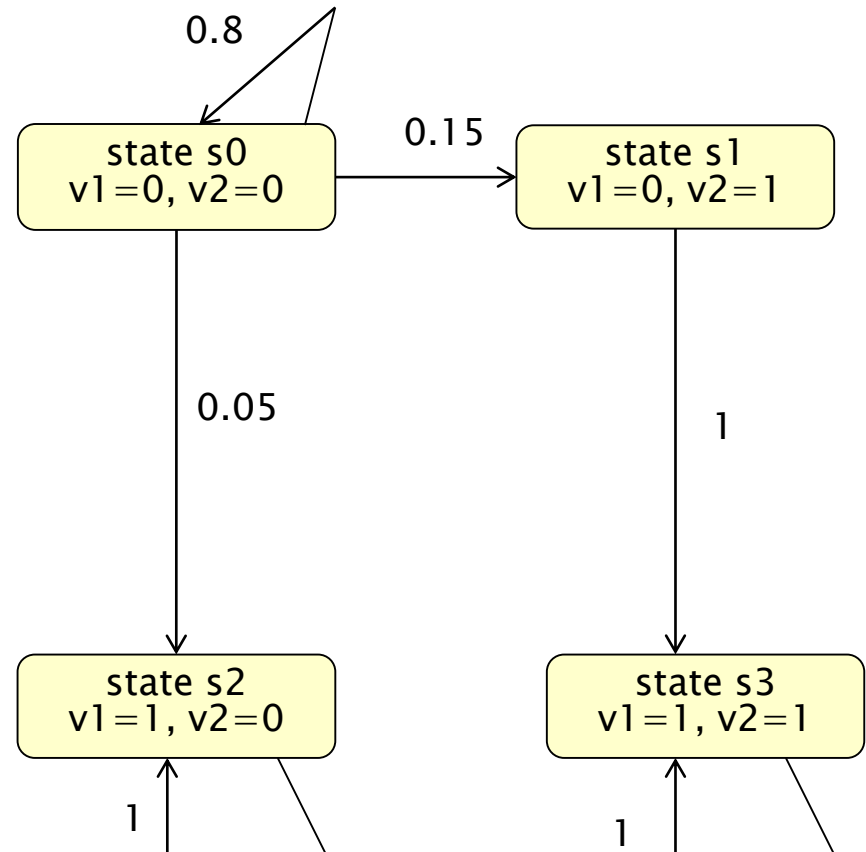
PRISM modelling language

- **Parallel composition**
 - model multiple components that can execute independently
 - for DTMC models, mostly assume components operate synchronously, i.e. move in lock-step
- **Synchronisation**
 - simultaneous transitions in more than one module
 - guarded commands with matching action-labels
 - probability of combined transition is product of individual probabilities for each component
- **More complex parallel compositions can be defined**
 - using process-algebraic operators
 - other types of parallel composition, action hiding/renaming
- **Required reading**
 - **The PRISM Language – Semantics** (available from <http://www.prismmodelchecker.org/doc/semantics.pdf>)

Example: DTMC described in PRISM

```
dtmc
module M1
  v1 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.9:(v1'=0) + 0.1:(v1'=1);
  [a] v1=0 & v2=1 -> 1:(v1'=1);
  [b] v1=1 -> 1:true;
endmodule
module M2
  v2 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.7:(v2'=0) + 0.3:(v2'=1);
  [a] v1=0 & v2=1 -> 1:true;
  [b] v1=1 -> 1:true;
endmodule
```

States and transitions generated by PRISM



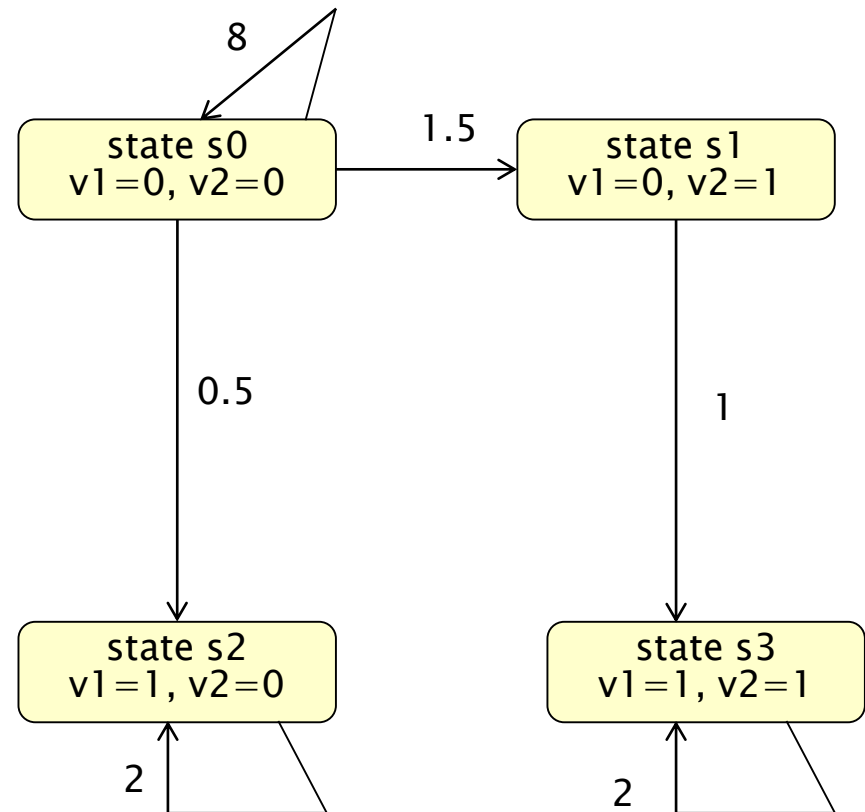
- **true** denotes an update where no variable's value changes
 - For example, in module M1 one could replace **true** with $(v1'=v1)$

- **Remark** Each state has a **probability distribution**. For example, in state s0:
$$0.8 + 0.15 + 0.05 = 1$$

Example: CTMC described in PRISM

```
ctmc
module M1
  v1 : [0..1] init 0;
  [] v1=0 & v2=0 -> 4.5:(v1'=0) + 0.5:(v1'=1);
  [a] v1=0 & v2=1 -> 1:(v1'=1);
  [b] v1=1 -> 1:true;
endmodule
module M2
  v2 : [0..1] init 0;
  [] v1=0 & v2=0 -> 3.5:(v2'=0) + 1.5:(v2'=1);
  [a] v1=0 & v2=1 -> 1:true;
  [b] v1=1 -> 2:true;
endmodule
```

States and transitions generated by PRISM



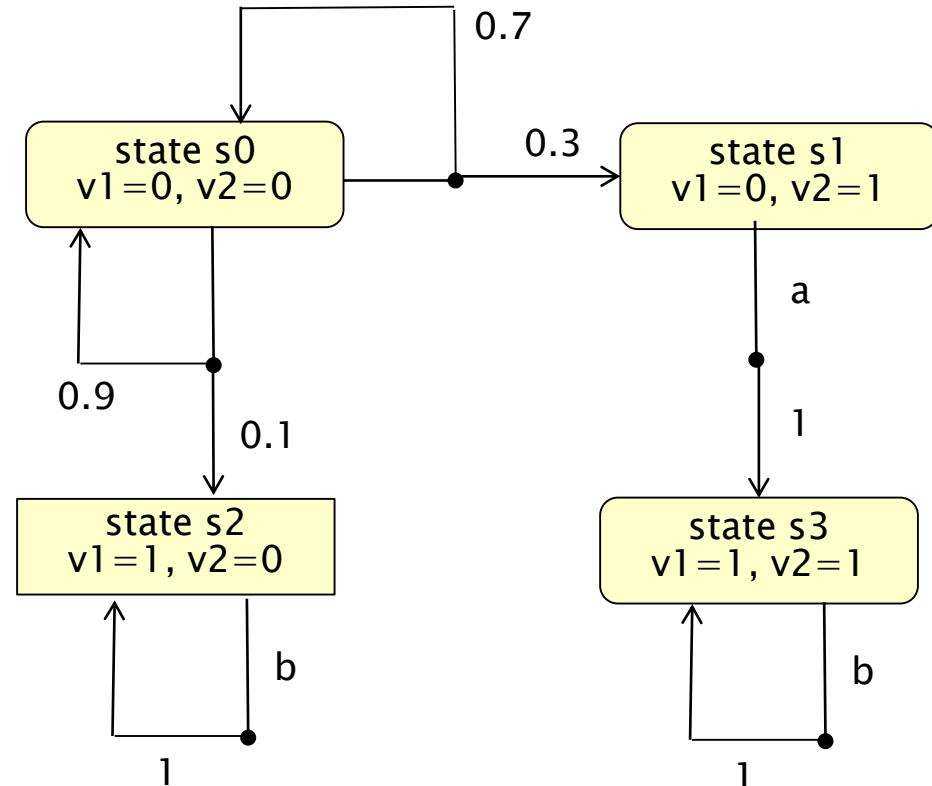
- In a CTMC updates are labeled with **rates** rather than probabilities
 - A rate is a positive real number ($\in \mathbb{R}_{\geq 0}$)
 - Computations proceed in continuous real time
 - Rates are used as parameters of the **exponential distribution**
 - The choice between transitions is modeled as a “race” between alternatives

Example: MDP described in PRISM

```
mdp
module M1
  v1 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.9:(v1'=0) + 0.1:(v1'=1);
  [a] v1=0 & v2=1 -> 1:(v1'=1);
  [b] v1=1 -> 1:true;
endmodule

module M2
  v2 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.7:(v2'=0) + 0.3:(v2'=1);
  [a] v1=0 & v2=1 -> 1:true;
  [b] v1=1 -> 1:true;
endmodule
```

States and transitions generated by PRISM



- A MDP can model both **probabilistic** and **nondeterministic** behavior
 - Example: the two unlabeled actions (that exit from state s0) are selected in a nondeterministic manner
 - Each action in a MDP has a probability distribution

Discrete-Time Markov Chains

- Formally, a DTMC D is a tuple $(S, s_{\text{init}}, P, L)$ where:
 - S is a set of states (“state space”)
 - $s_{\text{init}} \in S$ is the initial state
 - $P : S \times S \rightarrow [0,1]$ is the **transition probability matrix**
 where $\sum_{s' \in S} P(s, s') = 1$ for all $s \in S$
 - $L : S \rightarrow 2^{\text{AP}}$ is function labelling states with atomic propositions (taken from a set AP)

- Example:**

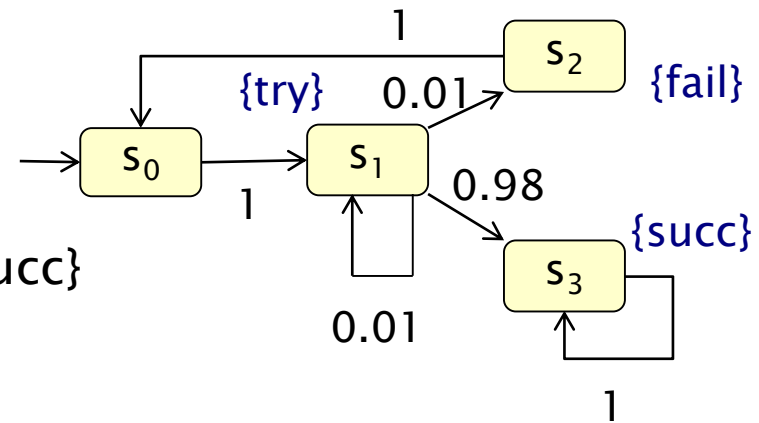
$$D = (S, s_{\text{init}}, P, L)$$

$$S = \{s_0, s_1, s_2, s_3\}$$

$$s_{\text{init}} = s_0$$

$$P = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0.01 & 0.01 & 0.98 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$\text{AP} = \{\text{try}, \text{fail}, \text{succ}\}$
 $L(s_0) = \emptyset,$
 $L(s_1) = \{\text{try}\},$
 $L(s_2) = \{\text{fail}\},$
 $L(s_3) = \{\text{succ}\}$



Remarks

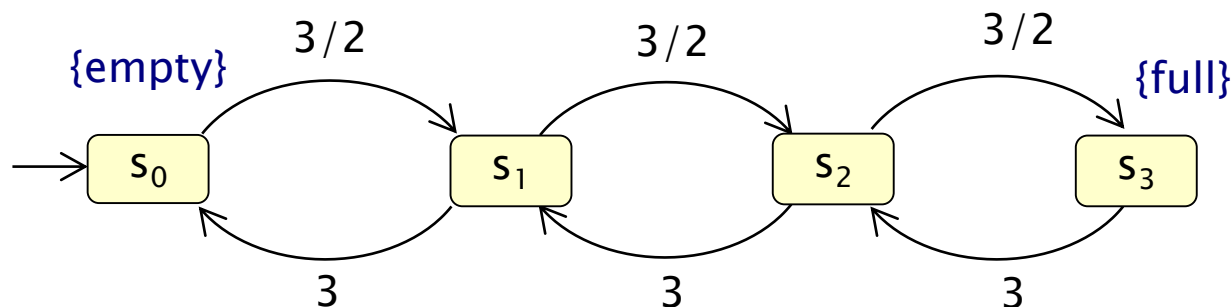
- We omitted atomic propositions in previous examples
- PRISM offers a keyword (**label**) for specifying atomic propositions

Continuous-Time Markov Chains

- Formally, a CTMC C is a tuple $(S, s_{\text{init}}, R, L)$ where:
 - S is a finite set of states (“state space”)
 - $s_{\text{init}} \in S$ is the initial state
 - $R : S \times S \rightarrow \mathbb{R}_{\geq 0}$ is the **transition rate matrix**
 - $L : S \rightarrow 2^{\text{AP}}$ is a labelling with atomic propositions
- Transition rate matrix assigns rates to each pair of states
 - used as a parameter to the **exponential distribution**

Simple CTMC example – modelling a queue of jobs:

- initially the queue is empty, maximum size of the queue is 3
- jobs **arrive** with rate $3/2$ (i.e. mean inter-arrival time is $2/3$)
- jobs are **served** with rate 3 (i.e. mean service time is $1/3$)
- state space: $S = \{s_i\}_{i=0..3}$ where s_i indicates i jobs in queue



Race conditions in CTMCs

- What happens when there exists **multiple** s' with $R(s,s') > 0$?
 - **Race condition**: first transition triggered determines next state. Two questions:
 1. How long is spent in a state s before a transition occurs?
 2. Which transition is eventually taken?

1. Time spent in a state before a transition

- Exponential distribution with parameter given by summation: $E(s) = \sum_{s' \in S} R(s,s')$
- Probability of leaving a state s within $[0,t]$ is: $1 - e^{-E(s) \cdot t}$
- $E(s)$ is the **exit rate** of state s ; s is called **absorbing** if $E(s)=0$ (no outgoing transitions)

2. Which transition is taken from state s ?

- Markov property: the choice is **independent** of the time at which it occurs
- Probability that next state from s is s' given by $P^{\text{emb}(C)}(s,s')$, computed based on
- The **embedded DTMC**: $\text{emb}(C) = (S, s_{\text{init}}, P^{\text{emb}(C)}, L)$

- state space, initial state and labelling as the CTMC

- for any $s, s' \in S$

$$P^{\text{emb}(C)}(s,s') = \begin{cases} R(s,s') / E(s) & \text{if } E(s) > 0 \\ 1 & \text{if } E(s) = 0 \text{ and } s = s' \\ 0 & \text{otherwise} \end{cases}$$

Two interpretations of a CTMC

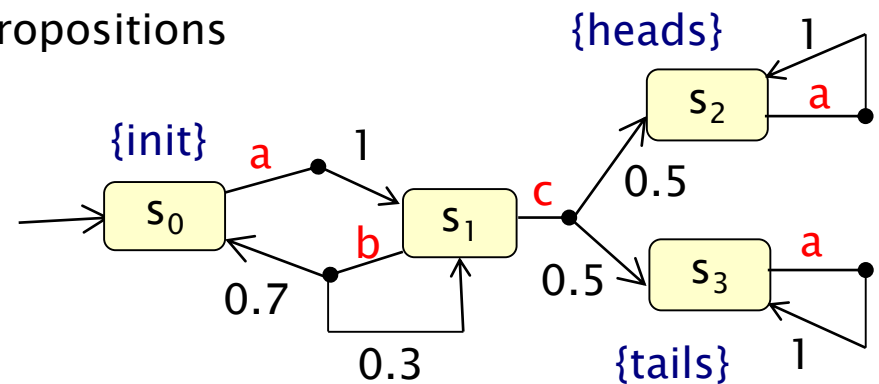
- Consider a (non-absorbing) state $s \in S$ with multiple outgoing transitions, i.e. multiple $s' \in S$ with $R(s,s') > 0$
- 1. Race condition
 - Each transition triggered after exponentially distributed delay
 - i.e. probability triggered before t time units: $1 - e^{-R(s,s') \cdot t}$
 - First transition triggered determines the next state
- 2. Separate delay/transition
 - Remain in s for delay exponentially distributed with rate $E(s)$
 - i.e. probability of taking an outgoing transition from s within $[0,t]$ is given by $1 - e^{-E(s) \cdot t}$
 - Probability that next state is s' is given by $P^{\text{emb}(C)}(s,s')$,
 - i.e., $R(s,s')/E(s) = R(s,s') / \sum_{s' \in S} R(s,s')$

Markov Decision Processes

- Formally, an MDP M is a tuple $(S, s_{\text{init}}, \text{Steps}, L)$ where:
 - S is a finite set of states (“state space”)
 - $s_{\text{init}} \in S$ is the initial state
 - Steps** : $S \rightarrow 2^{\text{Act} \times \text{Dist}(S)}$ is the **transition probability function**
 - Act is a set of actions and $\text{Dist}(S)$ is the set of discrete probability distributions over the set S
 - A (discrete) **probability distribution** over a countable set S is a function $\mu: S \rightarrow [0,1]$ satisfying $\sum_{s \in S} \mu(s) = 1$.
 - Notation:** We use $[s_0 \rightarrow p_0, \dots, s_n \rightarrow p_n]$ to denote the probability distribution that chooses s_i with probability p_i (for $1 \leq i \leq n$)
 - $L : S \rightarrow 2^{\text{AP}}$ is a labelling with atomic propositions

- Notes:**

- Steps(s) is always non-empty, i.e. no deadlocks
- The use of actions to label distributions is optional



PRISM property specification language

- Based on (probabilistic extensions of) temporal logic
 - Incorporates (subsumes) PCTL, CSL, LTL, PCTL* + costs/rewards
- Probabilistic operators P , S and R (see <http://www.prismmodelchecker.org/manual/>)
 - $P \sim p$ [pathprop] ($\in \text{Bool}$)
 - $\sim \in \{>=, >, <=, <\}$, $p \in [0,1]$ is a PRISM double
 - $>=p, >p / <=p, <p$ computes min./max. probabilities for MDPs and PTAs (over all adversaries, i.e. all possible resolutions of nondeterminism)
 - pathprop ::= X prop | prop1 U prop2 | F prop | ...
 - With (upper) “bounded” variants: prop1 $U \leq t$ prop2, $F \leq t$ prop
 - Lower bounds & time intervals: $F [t_1, t_2]$ prop ($t_1, t_2 \in \mathbb{R}_{\geq 0}$, only for CTMCs)
 - $S \sim p$ [prop] ($\in \text{Bool}$), for DTMC and CTMC
 - $R \sim r$ [rewardprop] ($\in \text{Bool}$), $r (\in \mathbb{R}_{\geq 0})$ is a PRISM double
 - rewardprop ::= F prop | $C \leq t$ | $I = t$ | S | ...
- Plus quantitative variants
 - $P=?$ [pathprop], $S=?$ [prop] ($\in [0,1]$) return the actual probability
 - $R=?$ [rewardprop] ($\in \mathbb{R}_{\geq 0}$) returns the actual expected reward
 - $=?$ replaced with $\min=?$ or $\max=?$ for MDPs or PTAs

Case Study: Dynamic Power Management

- Aims:
 - Low power consumption
 - Acceptable level of service
- We consider the PRISM solution available at
<http://www.prismmodelchecker.org/courses/pmc1112/>
<http://www.prismmodelchecker.org/tutorial/>
- PRISM modules:
 1. **Service (request) queue (SQ)**
 - q = current size of SQ
 - $q = 0 \Leftrightarrow$ empty queue
 2. **Service provider (SP)** – serves requests from SQ
 1. Power states: **sleep**, **idle**, **busy**
 3. **Power management (PM)** – simple PM policy:
 - $(q \geq q_trigger) \rightarrow$ sleep2idle
 - $(q=0) \rightarrow$ idle2sleep
- The technical solution is based on (a PM model for a Fujitsu disk drive given in)
 - Q. Qiu, Q. Wu, M. Pedram, “Stochastic modeling of a power-managed system: Construction and optimization”, Proc. International Symposium on Low Power Electronics and Design, pages 194--199, ACM Press, 1999

Case Study: Dynamic Power Management

- Properties specified by using the operators P and S:

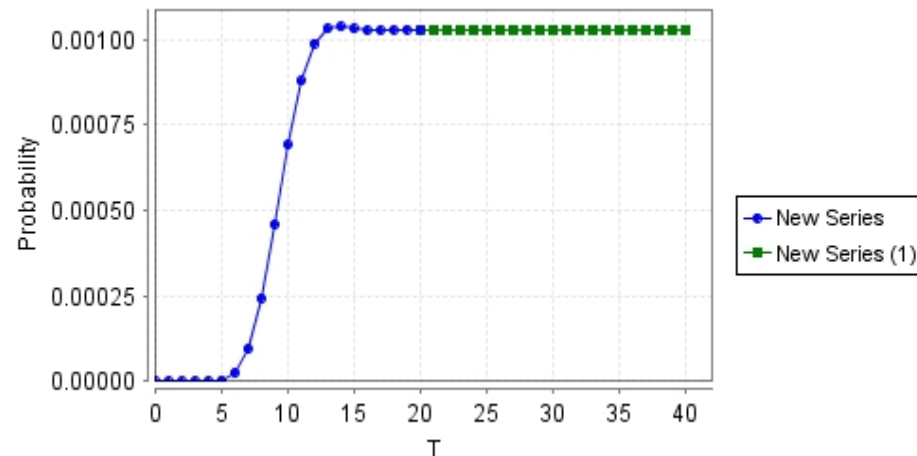
```
const double T;
```

```
P=? [F [T,T] q=q_max] // (transient) probability that q=q_max (queue is full) at time T  
                        // particular case of time interval [t1,t2] with t1=t2 (in general t1 ≤ t2)
```

```
S=? [q=q_max] // (steady state) long run probability that q=q_max
```

- **Experiments:** ranges of parameters, e.g.

- $P=? [F [T,T] q=q_max] \Rightarrow$
 - For $q_trigger=5$
 - $T=0,\dots,20$ (first 20 seconds)
 - Next $T=20,\dots,40$
- $S=? [q=q_max] \Rightarrow 0.001287\dots$
 - $0.001287\dots$ matches the results on the graph
 - Obtained with Gauss–Seidel
 - Jacobi (default method) doesn't converge here
- Identify **patterns**, **trends**, **anomalies** in **quantitative** results



Case Study: Dynamic Power Management

- Operator R can be used to compute expected values based on reward structures
- A reward structure is a list of reward items

rewards "reward_label"

```
// Assigns reward1 to all states that satisfy guard1
guard1 : reward1;
// Assigns reward2 to any transition with action label
// action2 from a state that satisfies guard2
[action2] guard2 : reward2;
endrewards
```

Simple examples:

rewards "queue_size"

```
true : q;
endrewards
```

rewards "lost"

```
[request] q = q_max : 1;
endrewards
```

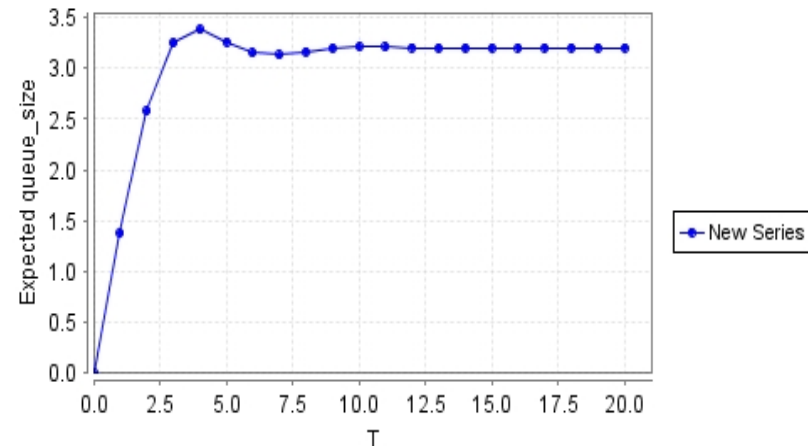
- Expected queue size at time T:

$R\{\text{"queue_size"}\}=? [I=T]$ $T = 0, \dots, 20 \Rightarrow$

- Long run expected (average) queue size:

$R\{\text{"queue_size"}\}=? [S] \Rightarrow 3.2038\dots$

3.2038... matches the results on the graph



More info on PRISM

- PRISM website: <http://www.prismmodelchecker.org/>
 - tool download: binaries, source code (GPL)
 - on-line example repository (50+ case studies)
 - on-line documentation:
 - PRISM manual
 - <http://www.prismmodelchecker.org/manual/>
 - PRISM tutorial
 - <http://www.prismmodelchecker.org/tutorial/>
 - <http://www.prismmodelchecker.org/courses/pmc1112/>
 - support: help forum, bug tracking, feature requests
 - related publications, talks, tutorials, links

References and recommended reading

1. D. Parker (DP), “Probabilistic Model Checking” – slides for a lecture course taught in the Department of Computer Science at Oxford University (2011)
<http://www.prismmodelchecker.org/lectures/pmc/>
2. Practical course (PRISM tutorial) accompanying [1]
<http://www.prismmodelchecker.org/tutorial/>
3. PRISM manual <http://www.prismmodelchecker.org/manual/>
4. Semantics of the PRISM language, available from
<http://www.prismmodelchecker.org/doc/semantics.pdf>
5. M. Kwiatkowska, G. Norman, D. Parker, “Stochastic model checking”, 2007
6. V. Forejt, M. Kwiatkowska¹, G. Norman, D. Parker, “Automated Verification Techniques for Probabilistic Systems”, 2011
7. G. Norman, D. Parker, J. Sproston, “Model Checking for Probabilistic Timed Automata”, 2013
8. M. Kwiatkowska, G. Norman, R. Segala, J. Sproston, “Automatic Verification of Real-time Systems with Discrete Probability Distributions”, 2002.
9. M. Kwiatkowska, G. Norman, J. Sproston, F. Wang, “Symbolic Model Checking for Probabilistic Timed Automata”, 2007
10. R. Alur, D. Dill, “A Theory of Timed Automata”, Theoretical Computer Science 126:183–235, 1994.